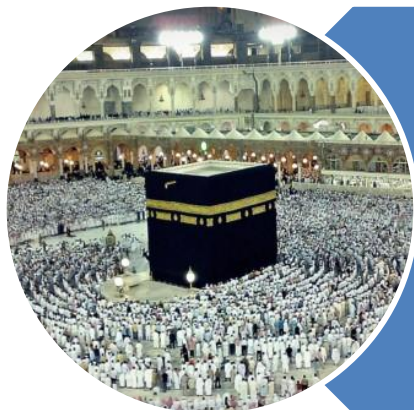


رَبِّ اشْرَحْ لِي صَدْرِي وَيَسِّرْ لِي أَمْرِي وَاحْلُلْ عُقْدَةً مِنْ لِسَانِي يَفْقَهُوا قَوْلِي



O my Lord! Expand for me my chest [with assurance] and ease for me my task and untie the knot from my tongue that they may understand my speech. **(Quran 20 : 25-28)**

QUEUES

Introduction

C

P

C

S

2

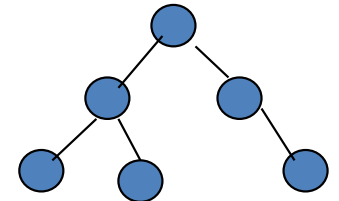
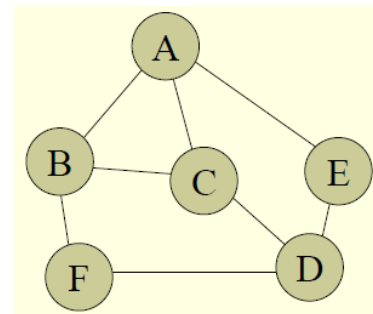
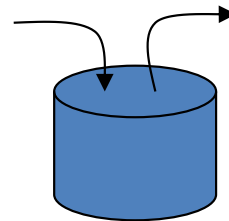
0

4

3

Queues - Introduction

- Arrays
- Linked List
- Stacks
- Queues
- Tree
- Graph



C

P

C

S

2

0

4

4

Queues - Introduction

- Definition

- A data structure that stores information in the form of a typical waiting line
- Placing a value one after the other

- Example

- Queue in Al-Baik
- Queue at Airport
- Queue at Bus Stop



C

P

C

S

2

0

4

5

Queues - Introduction

- Add to the **rear** of queue
- Remove from **front**
- Abstract Data Type (ADT)
- Logical data structure
- Implementation
 - Arrays
 - Linked List



C

P

C

S

2

0

4

6

Queues - Introduction

- **Access Policy**

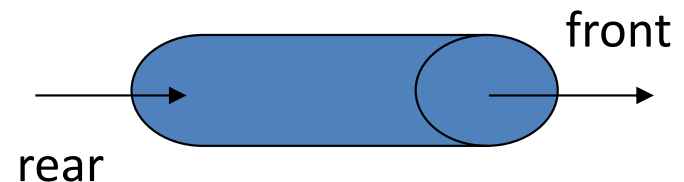
- The first element that is inserted into the queue is the first element that will leave the queue

- **First In First Out**

- FIFO

- **Processing End**

- Front (to remove)
- Rear (to add)



C

P

C

S

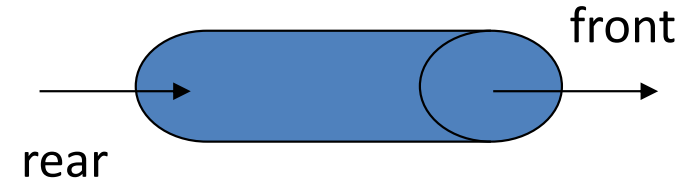
2

0

4

7

Queues – Summary



- Abstract Data Type (ADT)
- Logical Data Structure
- Linear Data Structure
- Homogeneous
- Accessible only from both ends i. e. front & rear
- Access policy is FIFO
- Implemented using Arrays or Linked List

QUEUES

Basic Operations
&
Its Use

C

P

C

S

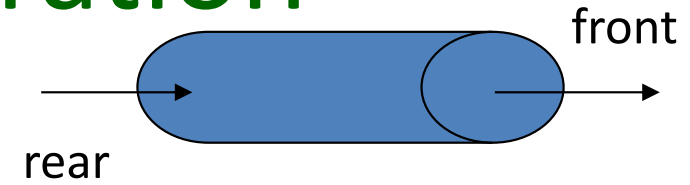
2

0

4

Queues – Basic Operation

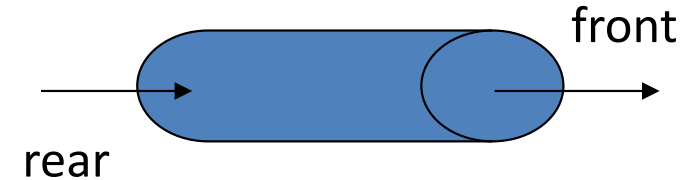
- enqueue
- dequeue
- peek
- isEmpty
- isFull
- clear



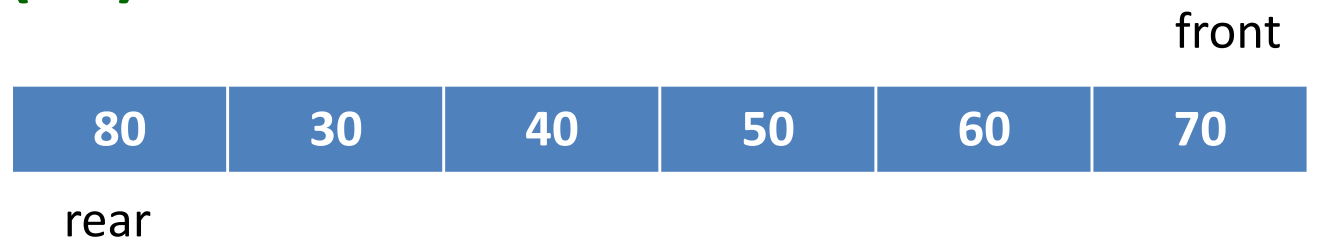
Queues – Basic Operation

- **enqueue**

- Inserts an element at the end of the queue
- **$O(1)$**



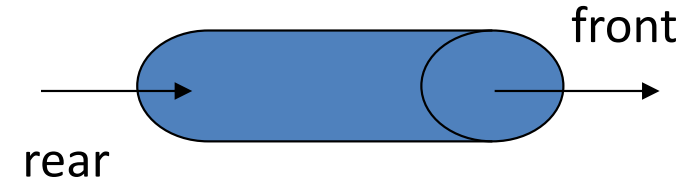
- **enqueue (80)**



Queues – Basic Operation

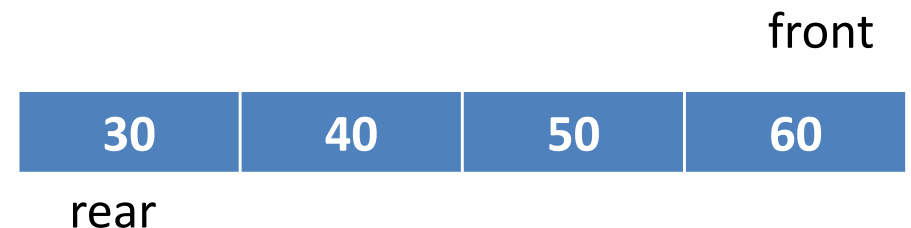
- **dequeue**

- Removes the element from the front of the queue
- **$O(1)$**



- **dequeue ()**

Value = 70

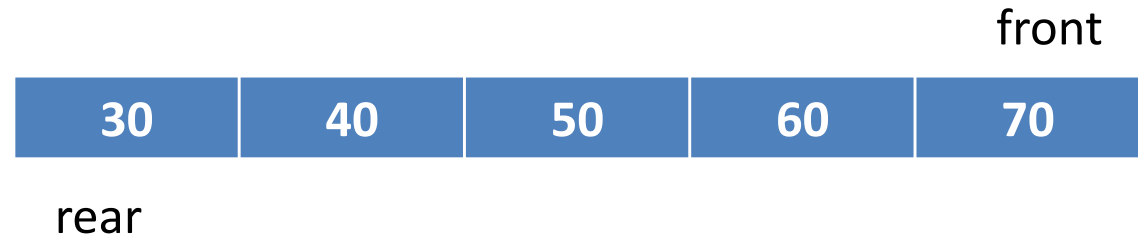
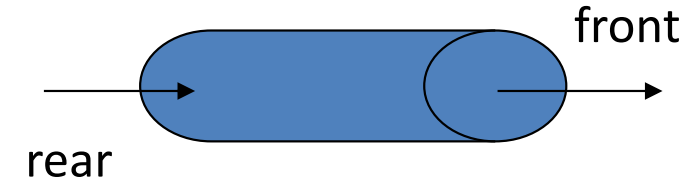


Queues – Basic Operation

- peek

- Return the element from the front of the queue without removing it

- $O(1)$



- `peek()`

Value = 70



C

P

C

S

2

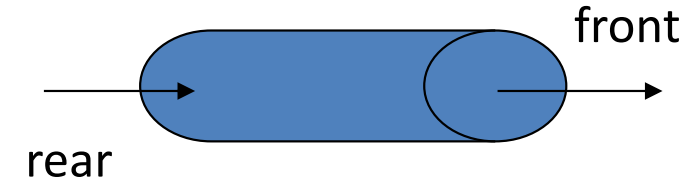
0

4

Queues – Basic Operation

- **isEmpty**

- Checks to see if the queue is empty
- **$O(1)$**



- **isEmpty ()**

count = 0

True

otherwise

False



C

P

C

S

2

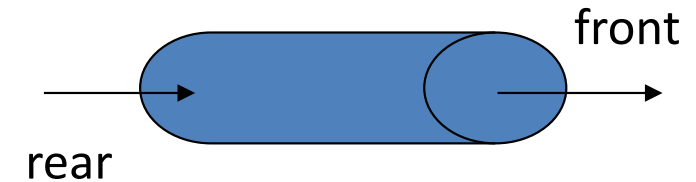
0

4

Queues – Basic Operation

- **isFull**

- Checks to see if the queue is full
- **$O(1)$**



- **isFull ()**



count = maxSize

True

otherwise

False

C

P

C

S

2

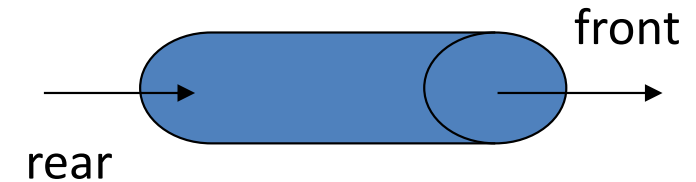
0

4

Queues – Basic Operation

- **clear**

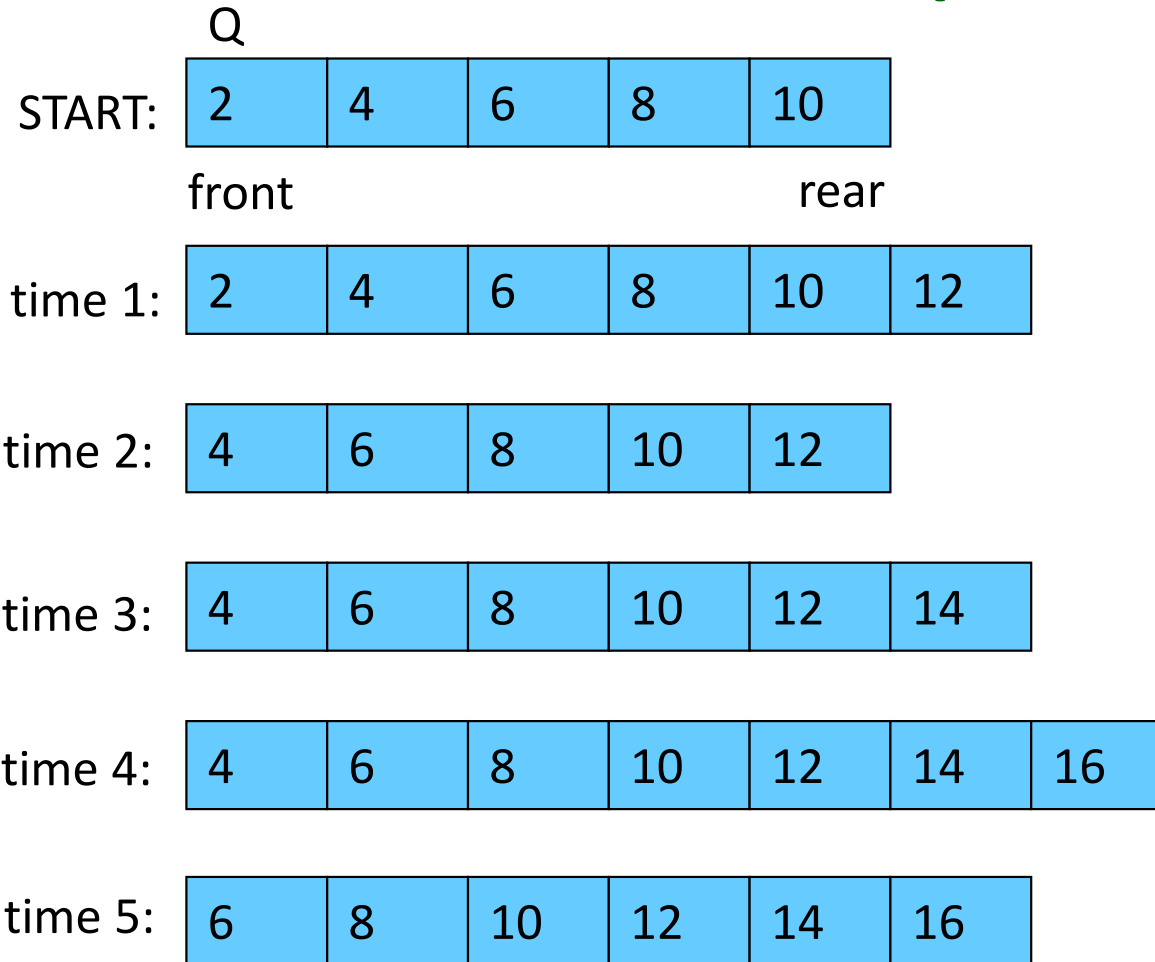
- Remove all values from the queue
- **$O(n)$**



- **clear ()**



Queues – Basic Operation



Sequence of operations

Time Operation

- 1 enqueue(12)
- 2 dequeue
- 3 enqueue(14)
- 4 enqueue(16)
- 5 dequeue

C

P

C

S

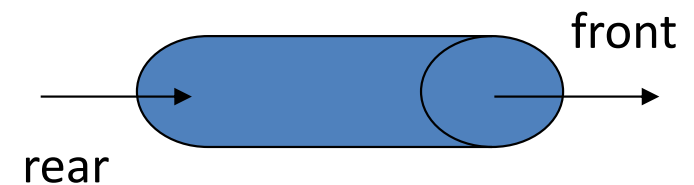
2

0

4

Queues – Basic Operation

- | | |
|-----------|--------------------|
| • enqueue | Modify the content |
| • dequeue | Modify the content |
| • peek | Do not modify |
| • isEmpty | Do not modify |
| • isfull | Do not modify |
| • clear | Modify the content |



QUEUES

Implementation Logic

“Brute Force” method

Front fixed & Rear moveable

C

P

C

S

2

0

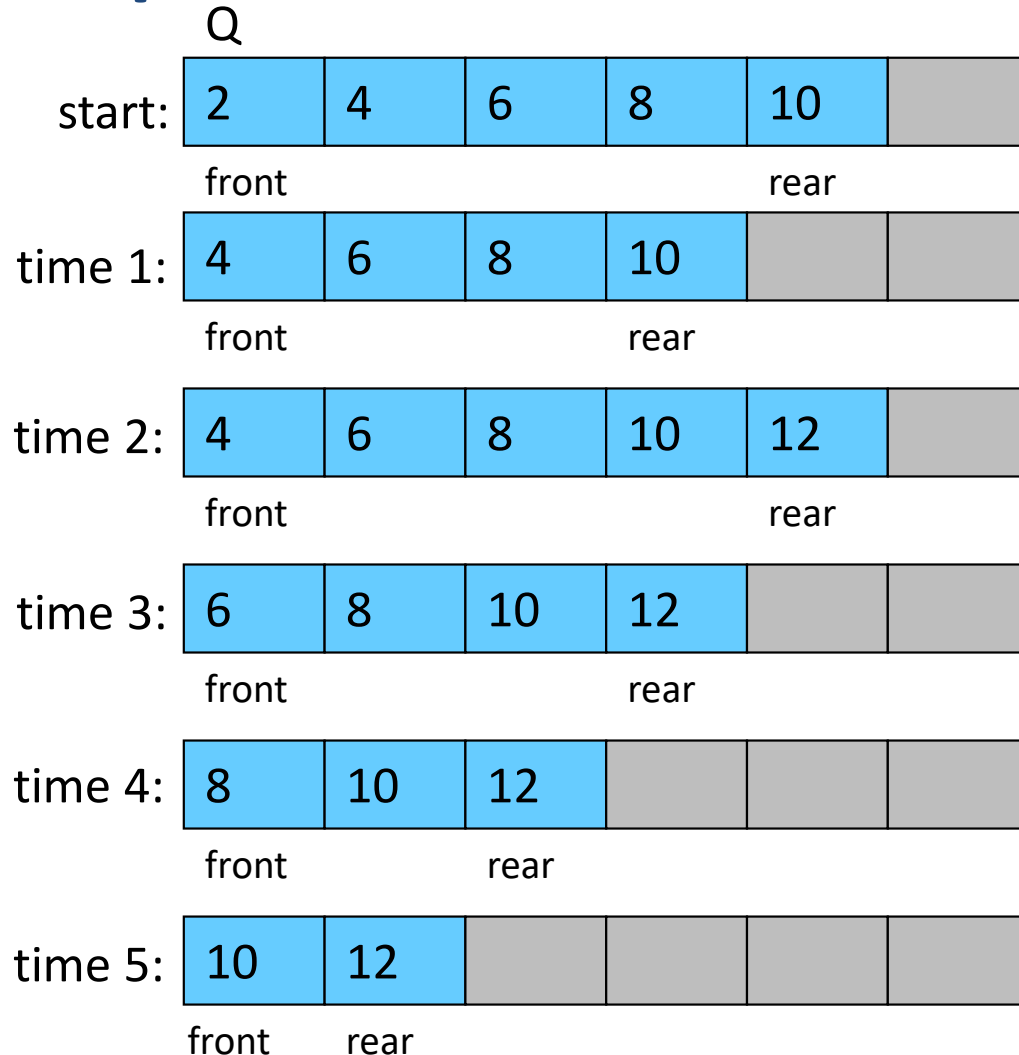
4

Implementation – “Brute Force” method

- Real Time approach
- Service desk doesn't move
 - After customer at front served
 - He will leave the queue
 - All customers have to take one step ahead



Implementation – “Brute Force” method



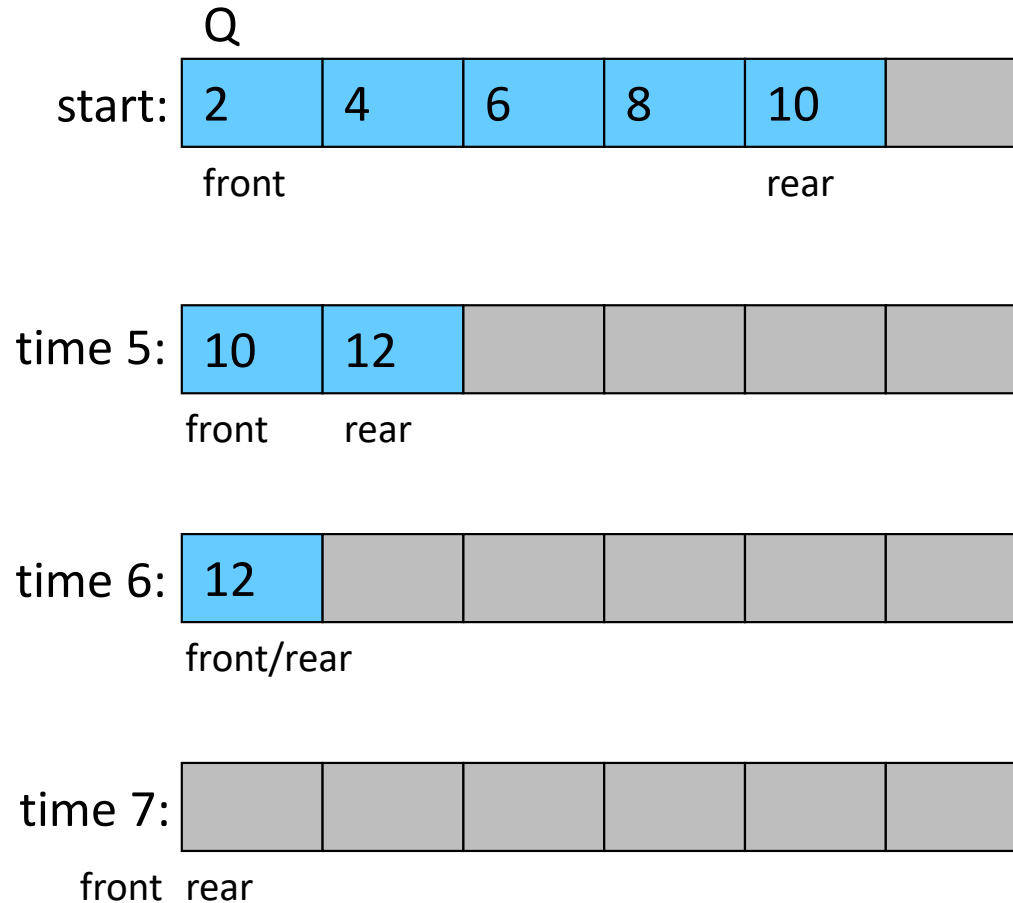
Sequence of operations

Time Operation

- 1 dequeue
- 2 enqueue(12)
- 3 dequeue
- 4 dequeue
- 5 dequeue
- 6 dequeue
- 7 dequeue

Notice that the array now has room to add elements to the queue.

Implementation – “Brute Force” method



Sequence of operations

Time Operation

- 1 dequeue
- 2 enqueue(12)
- 3 dequeue
- 4 dequeue
- 5 dequeue
- 6 dequeue
- 7 dequeue

C

P

C

S

2

0

4

Implementation – "Brute Force" method

- Insertion will be at rear

- No shifting required

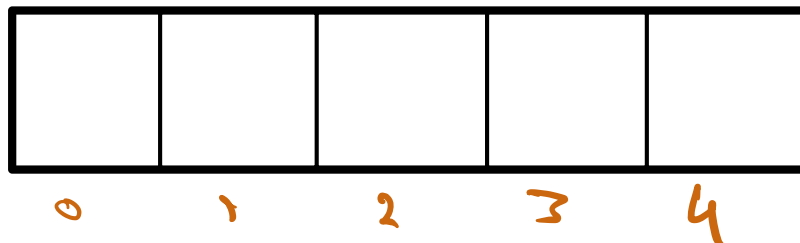
- $O(1)$



- Removal will be from front

- shifting required

- $O(n)$



“No Bonus” marks for this course anymore

C
P
C
S

2
0
4

QUEUES

Implementation Logic

“Unrealistic” method

Front & Rear moveable

C

P

C

S

2

0

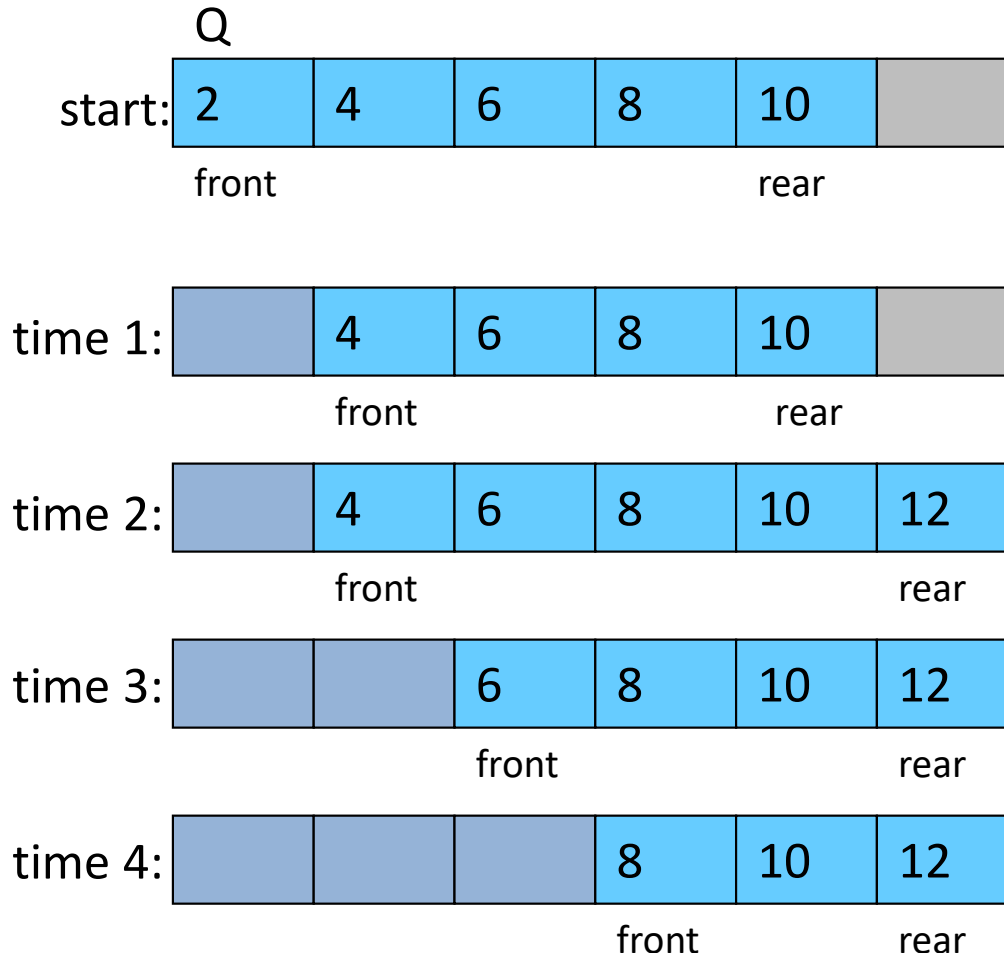
4

Implementation – “Unrealistic” method

- Unrealistic approach
- Make service desk moveable
 - After customer at front served
 - Service desk will move to next person
 - No need to move the customers



Implementation – “Unrealistic” method



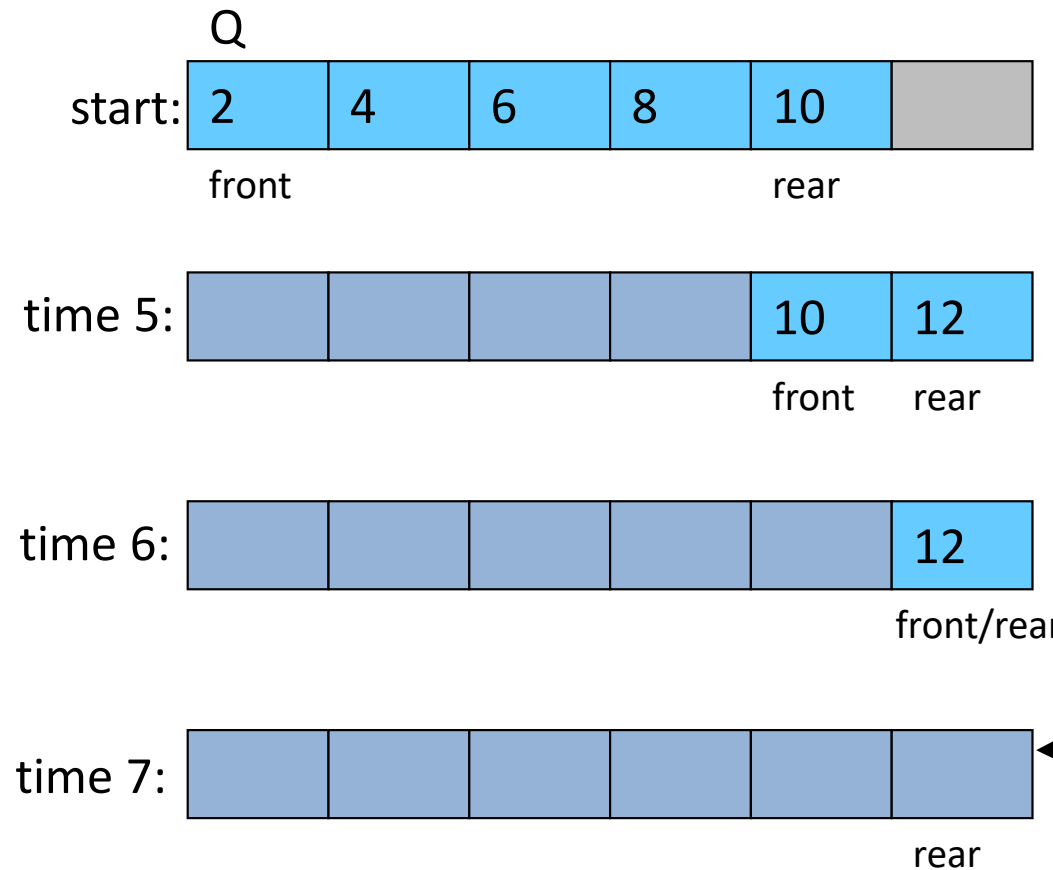
Sequence of operations

Time Operation

- 1 dequeue
- 2 enqueue(12)
- 3 dequeue
- 4 dequeue
- 5 dequeue
- 6 dequeue
- 7 dequeue

Notice that the queue now appears to be full even though there are locations available in the array!

Implementation – "Unrealistic" method



Sequence of operations

Time Operation

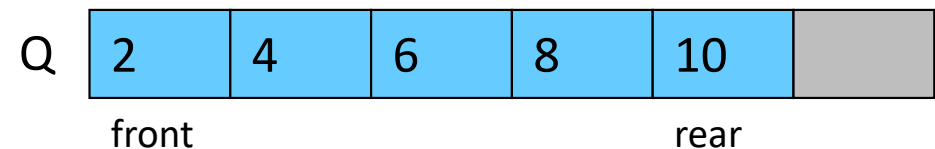
- 1 dequeue
- 2 enqueue(12)
- 3 dequeue
- 4 dequeue
- 5 dequeue
- 6 dequeue
- 7 dequeue

Even Worse:
The queue now appears full even though there are NO items in the array!

Implementation – “Unrealistic” method

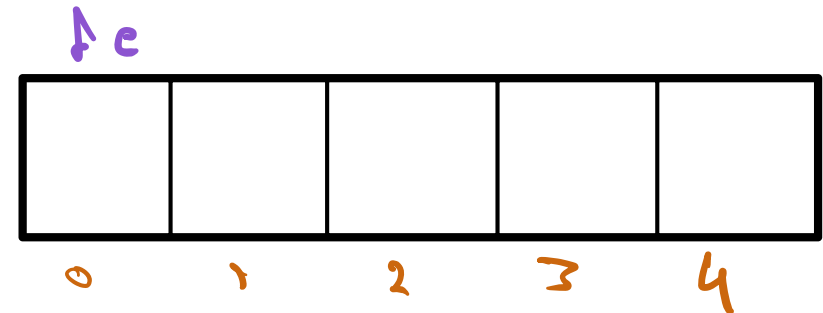
- Insertion will be at rear

- No shifting required
- Increment in rear
- $O(1)$



- Removal will be from front

- No shifting required
- Increment in front
- $O(1)$



- **Problem**

- Space utilization is not optimal
- Wasted cell

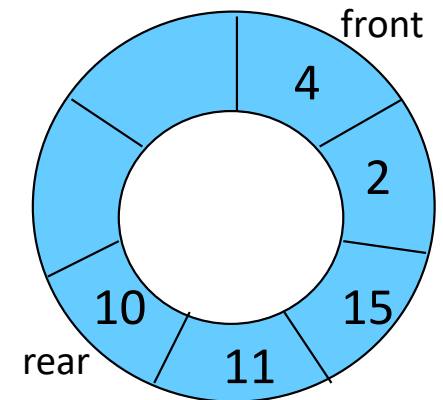
QUEUES

Implementation Logic

Circular Arrays

Implementation – using “Circular Array”

- Common way of implementing an array-based queue
- It is a regular array 
 - Increments will be made circular
 - It will behave like circular array
- Queue Empty
 - count = 0
- Queue Full
 - count = maxsize



visualization of
circular array

Implementation – using “Circular Array”

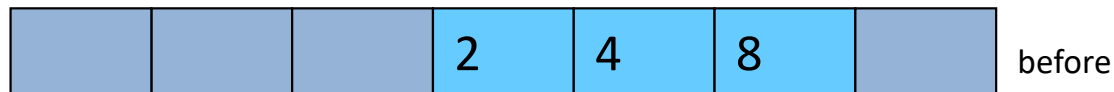


front rear



front rear

normal array implementation

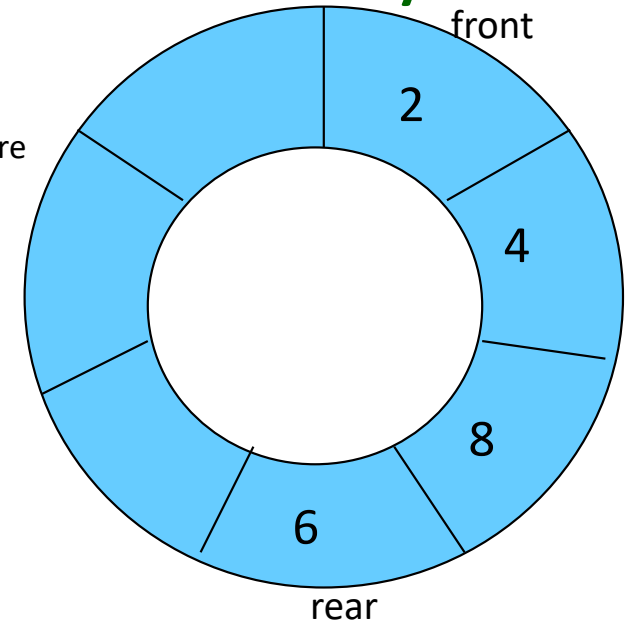


front rear



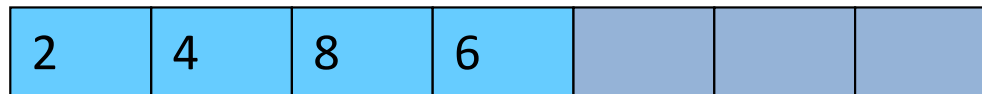
front rear

circular array implementation



visualization of a queue
implemented as a circular array
after insertion of element 6

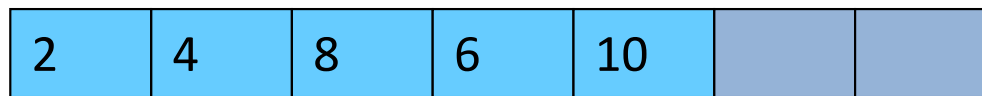
Implementation – using “Circular Array”



front

rear

before



front

rear

after

normal array implementation



front

rear

before

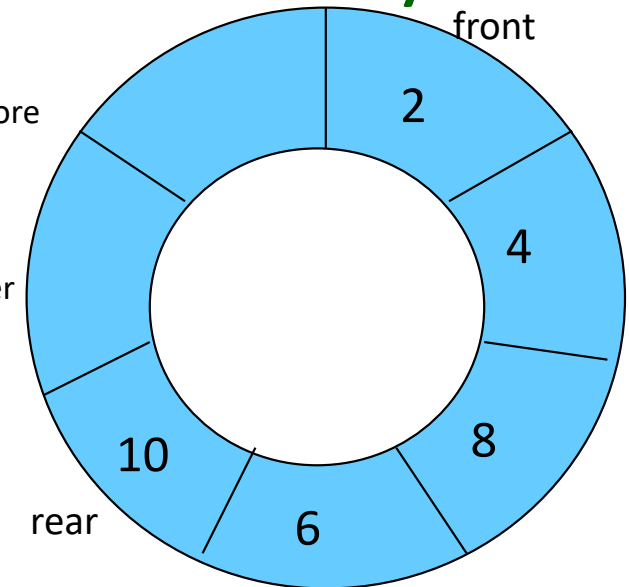


rear

front

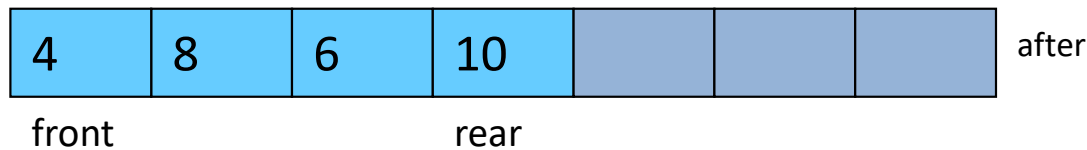
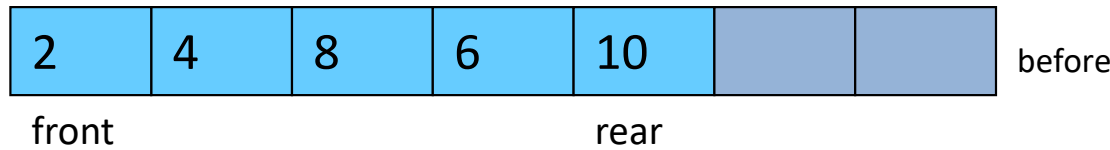
after

circular array implementation

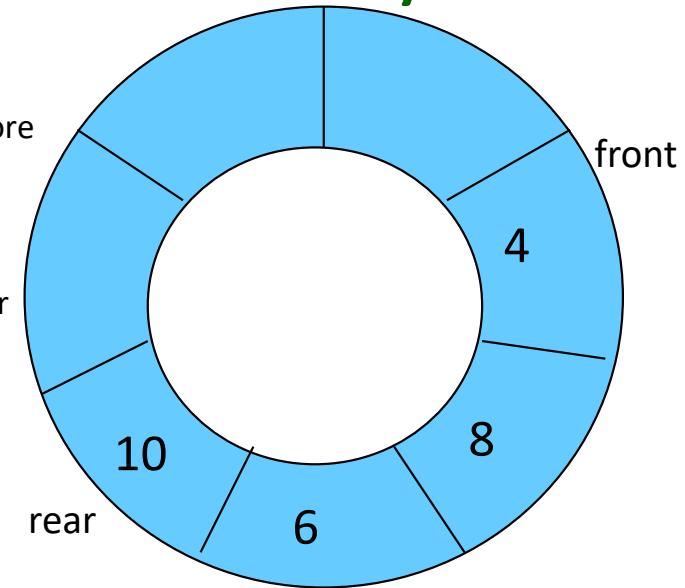


visualization of a queue
implemented as a circular array
after insertion of element 10

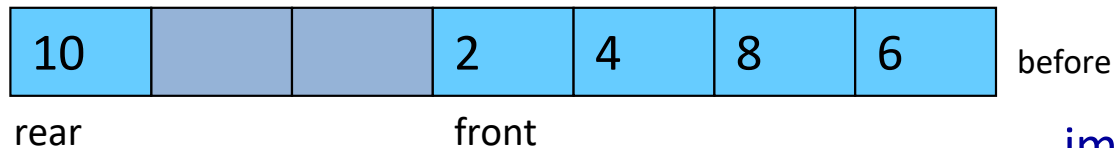
Implementation – using “Circular Array”



normal array implementation



before visualization of a queue implemented as a circular array after dequeue operation



circular array implementation

Implementation – using “Circular Array”



front

rear

before

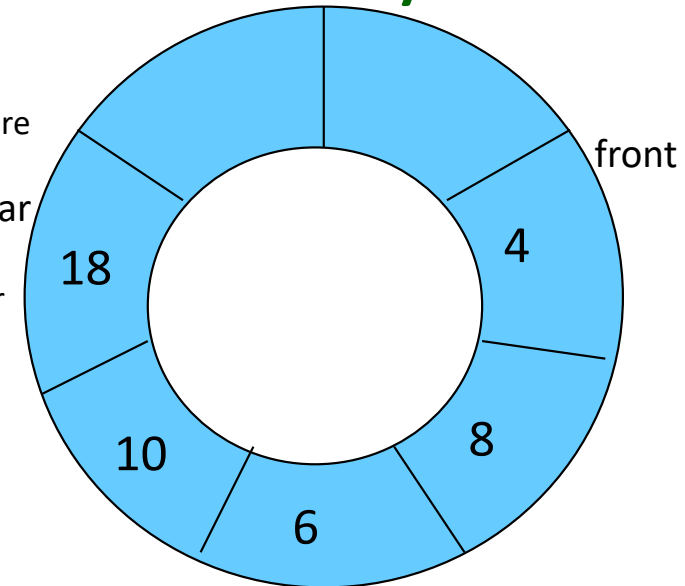


front

rear

after

normal array implementation



rear

front

before



rear

front

after

circular array implementation

visualization of a queue
implemented as a circular array
after insertion of element 18

Implementation – using “Circular Array”



front

rear

before



front

rear

after

normal array implementation



rear

front

before

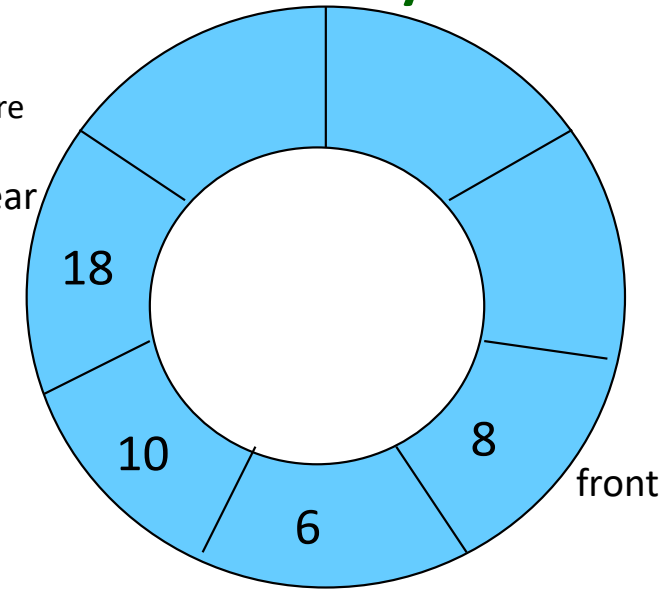


rear

front

after

circular array implementation



visualization of a queue
implemented as a circular array
after dequeue operation

Implementation – using “Circular Array”



front

rear

before

rear

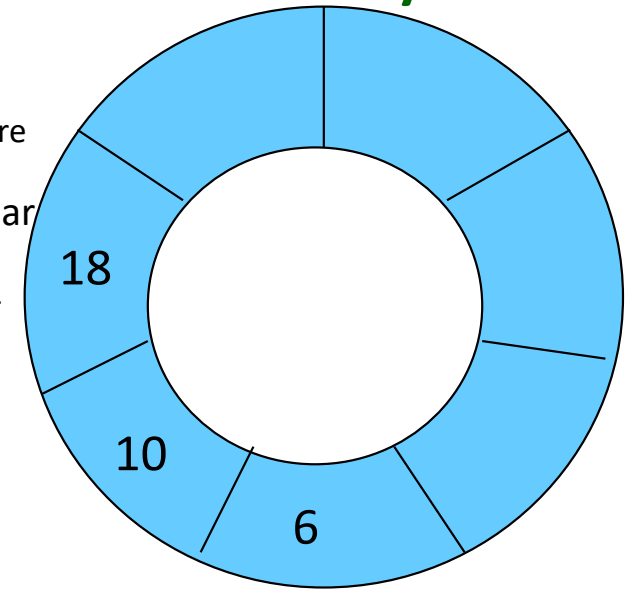


front

rear

after

normal array implementation



front



rear

front

before

visualization of a queue
implemented as a circular array
after dequeue operation



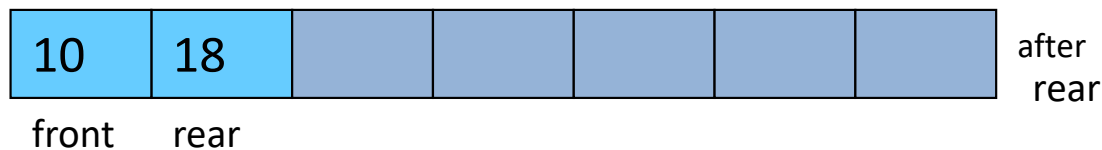
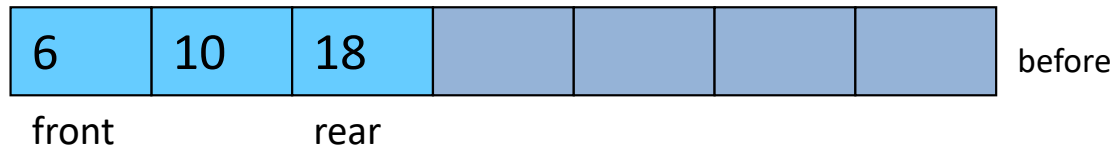
rear

front

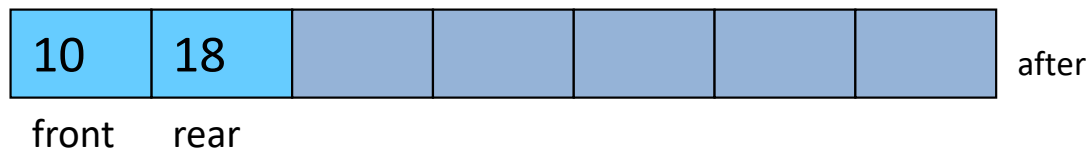
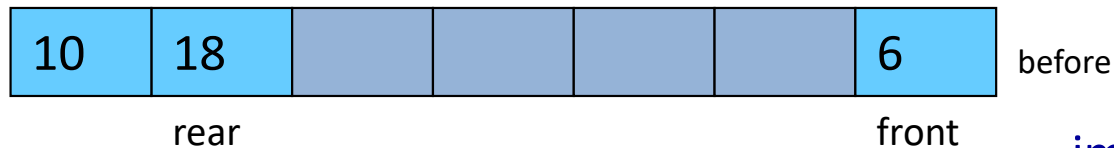
after

circular array implementation

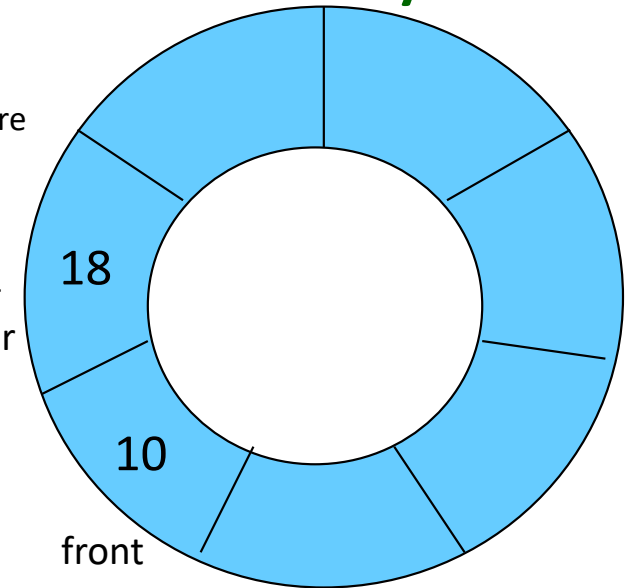
Implementation – using “Circular Array”



normal array implementation



circular array implementation



visualization of a queue
implemented as a circular array
after dequeue operation

Implementation – using “Circular Array”



front rear



front rear

normal array implementation

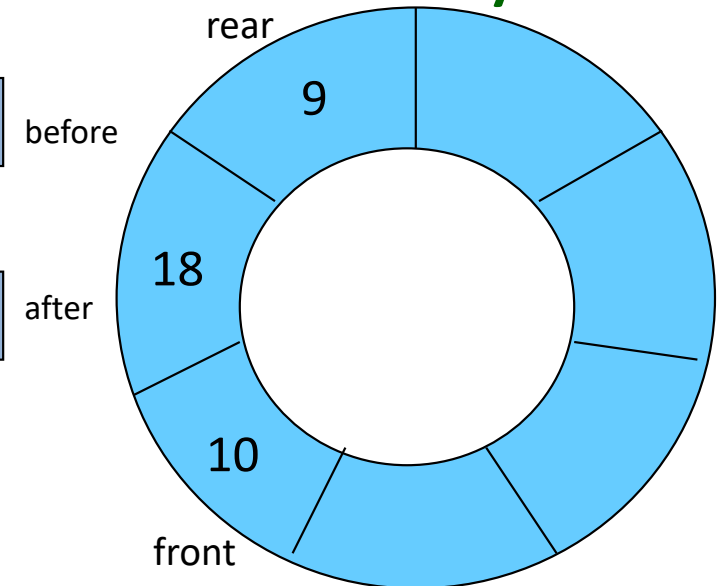


front rear



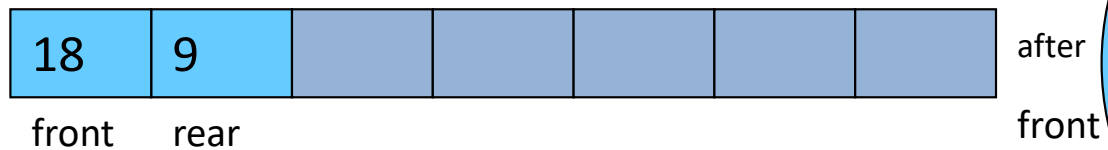
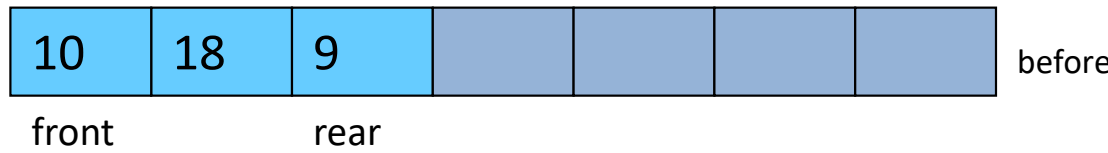
front rear

circular array implementation

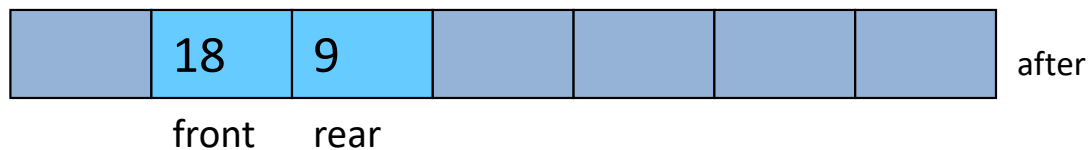
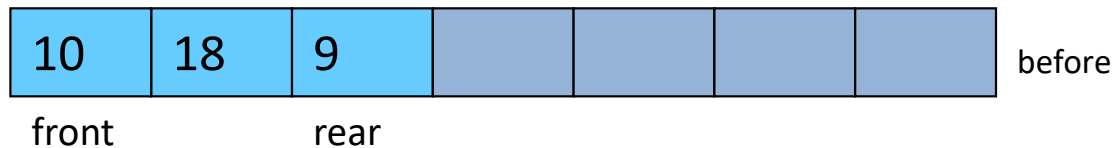


visualization of a queue
implemented as a circular array
after insertion of element 9

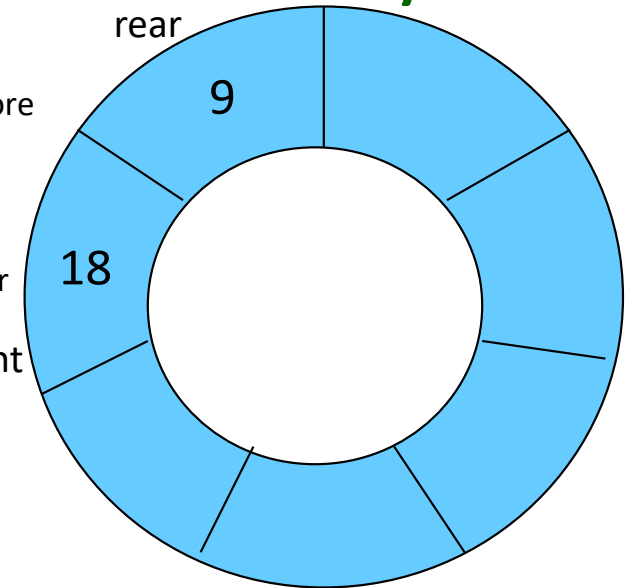
Implementation – using “Circular Array”



normal array implementation



circular array implementation



visualization of a queue
implemented as a circular array
after dequeue operation

Implementation – using “Circular Array”



front rear



front/rear

normal array implementation

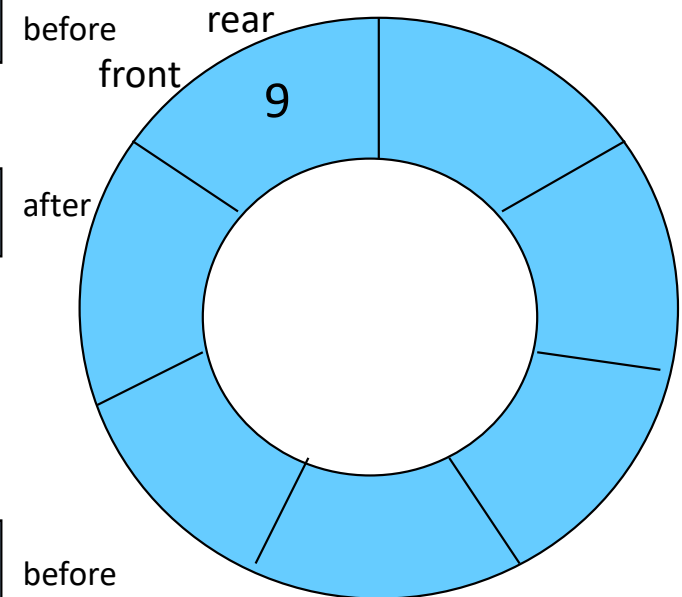


front rear



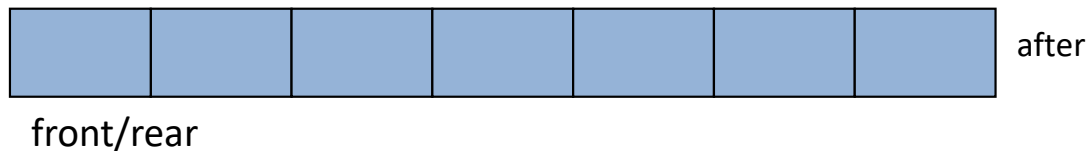
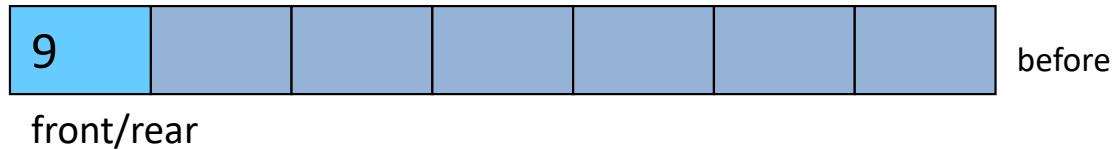
front/rear

circular array implementation

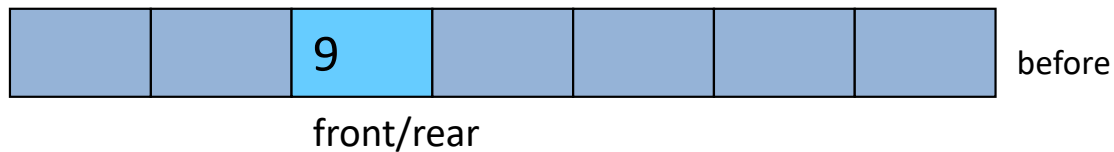


visualization of a queue
implemented as a circular array
after dequeue operation

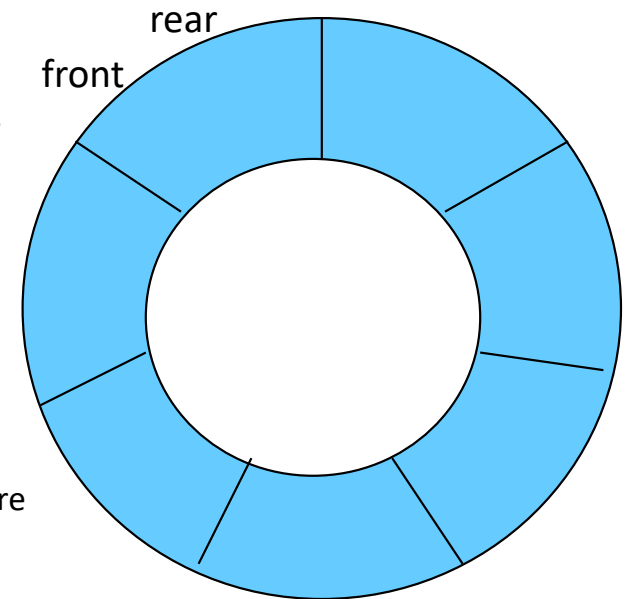
Implementation – using “Circular Array”



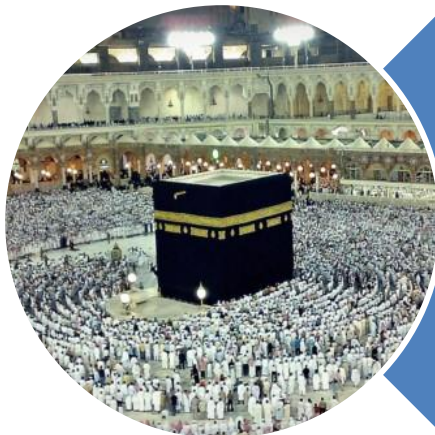
normal array implementation



circular array implementation



visualization of a queue
implemented as a circular array
after dequeue operation



Allah (Alone) is Sufficient for us,
and He is the Best Disposer of
affairs (for us). (Quran 3 : 173)